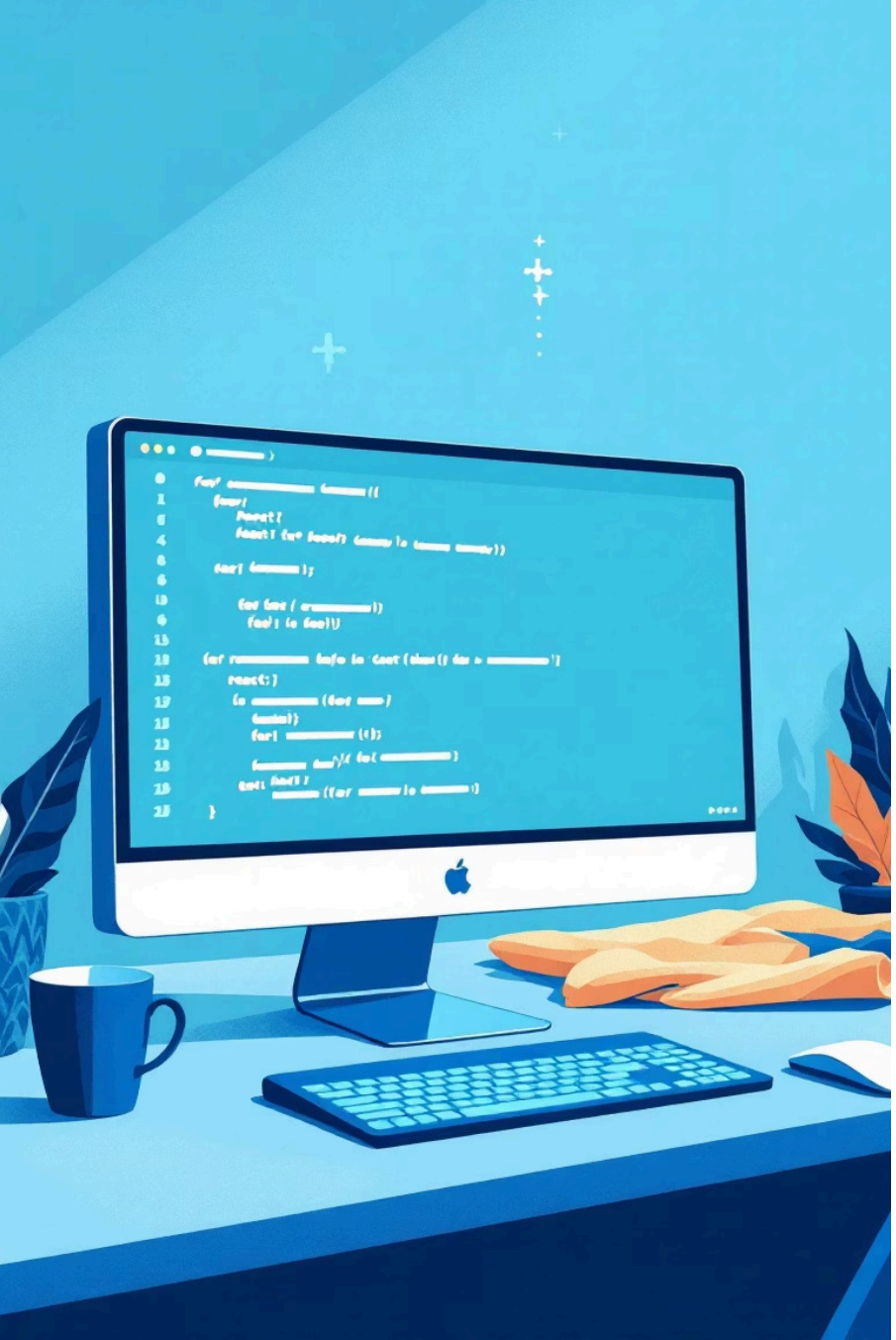


React.js: The Complete Developer's Guide

Master React from basics to advanced concepts with this comprehensive study guide. Transform your web development skills with practical examples, best practices, and real-world applications.

Made with GAMMA

What Makes React So Popular?

Component-Based Architecture

Build encapsulated components that manage their own state, then compose them to create complex UIs. Reusable, maintainable, and scalable.

Virtual DOM Performance

React creates a virtual representation of the DOM in memory. Changes are calculated efficiently and only necessary updates are applied to the real DOM.

Unidirectional Data Flow

Data flows down from parent to child components, making applications predictable and easier to debug. No two-way binding complexity.

React revolutionises UI development by making complex interfaces manageable through simple, predictable patterns that scale with your application's growth.

Made with GAMMA

Getting Started: Your First React Setup

Choose Your Setup Method

01

Create React App

Perfect for beginners. Zero configuration required.

```
npx create-react-app my-app
cd my-app
npm start
```

02

Vite (Recommended)

Lightning-fast build tool with modern features.

```
npm create vite@latest my-app --
--template react
```

03

Next.js

Full-stack React framework with built-in optimisations.

```
npx create-next-app@latest my-app
```

Pro Tip: Vite offers significantly faster development server startup and hot module replacement compared to Create React App.

Made with GAMMA

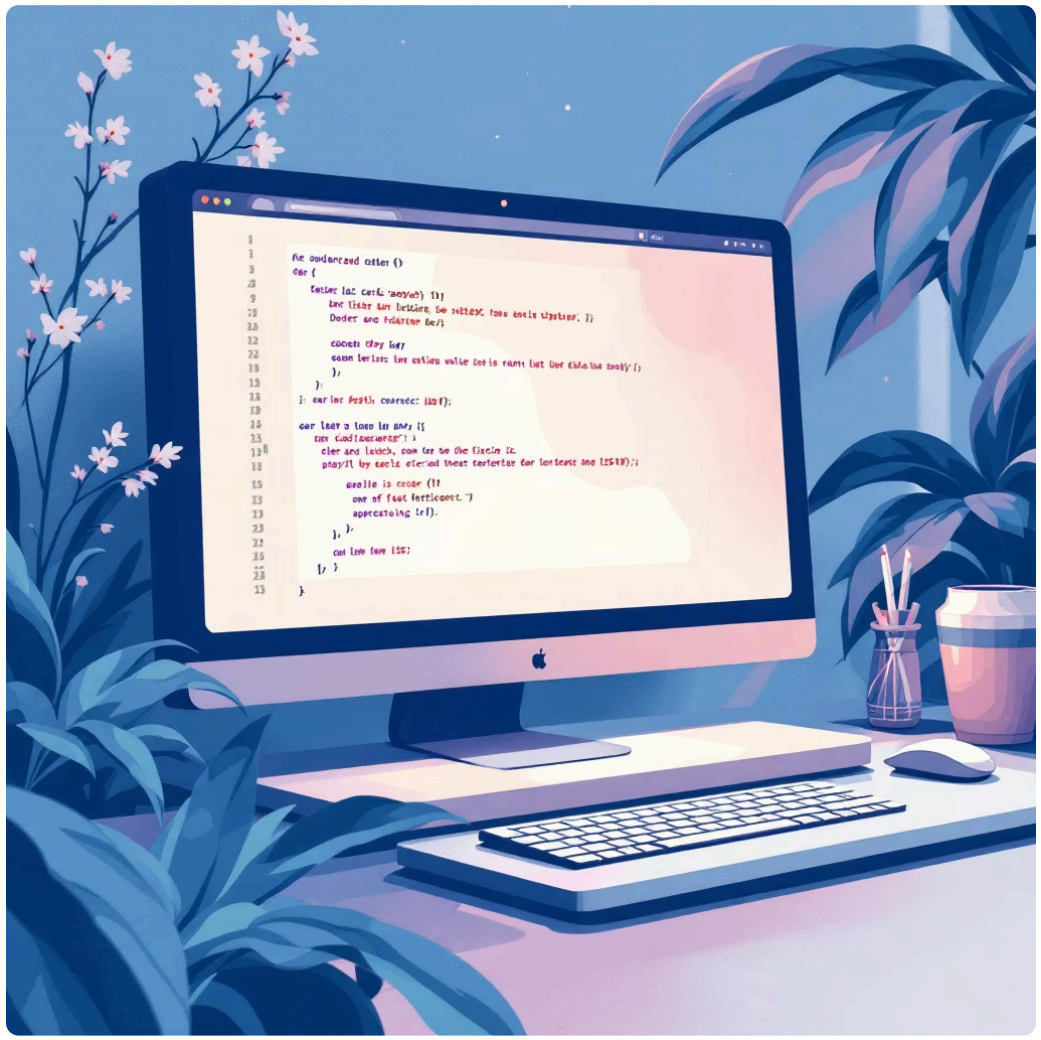
JSX: Writing HTML in JavaScript

JSX makes React components readable and expressive by allowing HTML-like syntax within JavaScript.

JSX Syntax Rules

- Elements must be wrapped in a single parent
- Use className instead of class
- Close all tags (including self-closing ones)
- Use camelCase for attributes

```
// Valid JSX
function Welcome() {
  return (
    <div className="container">
      <h1>Hello World!</h1>
      <img src="logo.png" alt="Logo" />
    </div>
  );
}
```



JavaScript Expressions

Embed dynamic content using curly braces:

```
const name = "Developer";
const element = <h1>Hello, {name}!</h1>;

// Dynamic attributes
<img src={user.avatar} alt={user.name} />
```

Made with GAMMA

Props & State: Data Management Fundamentals

Props (Properties)

Data passed from parent to child components. Read-only and help make components reusable.

```
function UserCard({ name, age, email }) {
  return (
    <div>
      <h3>{name}</h3>
      <p>Age: {age}</p>
      <p>Email: {email}</p>
    </div>
  );
}
```

State (useState)

Component's internal data that can change over time. Triggers re-renders when updated.

```
import { useState } from 'react';

function Counter() {
 const [count, setCount] = useState(0);

 return (
 <button onClick={() => setCount(count + 1)}>
 Count: {count}
 </button>
 );
}
```

Made with GAMMA

Essential React Hooks

1

useState

Manage component state

```
const [state, setState] = useState(initialValue);
```

2

useEffect

Handle side effects and lifecycle events

```
useEffect(() => {
  // Effect logic
  return () => {
    // Cleanup (optional)
  };
}, [dependencies]);
```

3

useRef

Access DOM elements directly

```
const inputRef = useRef(null);
// Access: inputRef.current
```

4

useMemo & useCallback

Optimise performance by memoizing values and functions

```
const memoizedValue = useMemo(() =>
 computeExpensiveValue(a, b), [a, b]);
```

Made with GAMMA

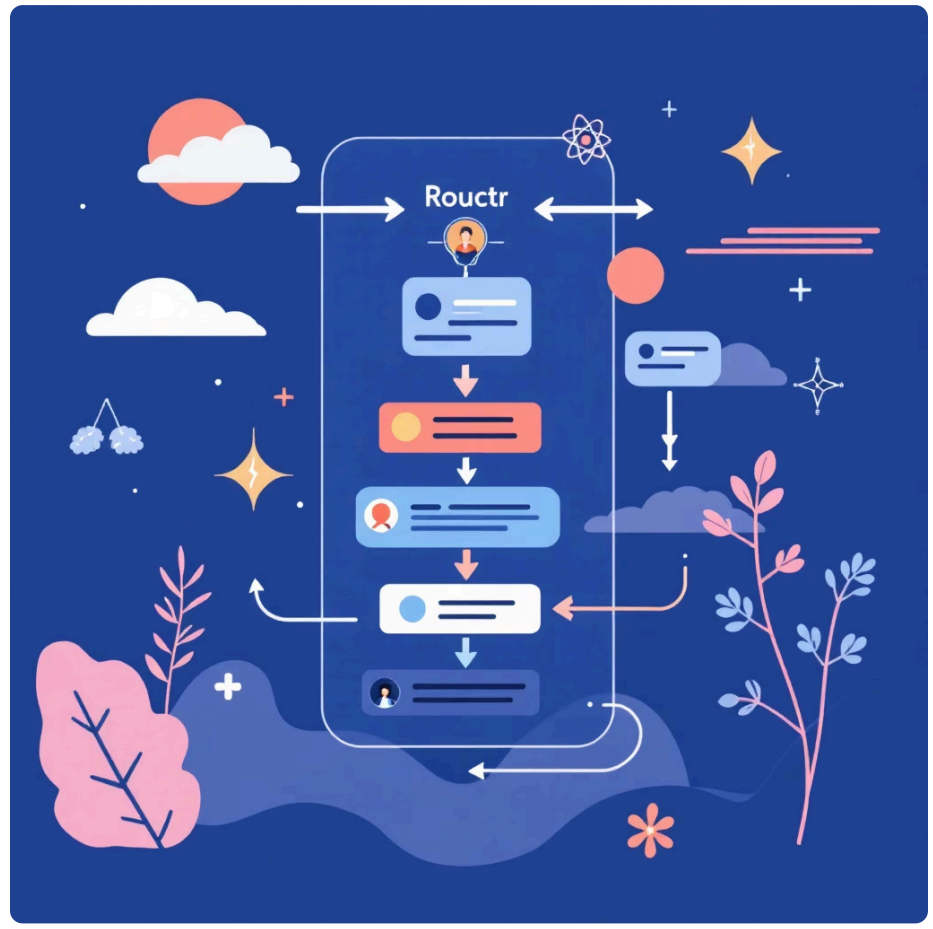
Navigation with React Router

Setting Up Routes

```
import { BrowserRouter, Routes, Route } from 'react-router-dom';

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/users/:id" element={<User />} />
      </Routes>
    </BrowserRouter>
  );
}
```

Navigation Components

```
import { Link, useNavigate } from 'react-router-dom';

// Declarative navigation
<Link to="/about">About Us</Link>

// Programmatic navigation
const navigate = useNavigate();
navigate('/dashboard');
```



Route Parameters: Access dynamic segments using useParams() hook to get URL parameters like :id in your components.

Made with GAMMA

State Management & Context API

Move beyond prop drilling with React's built-in state management solution.

Create Context

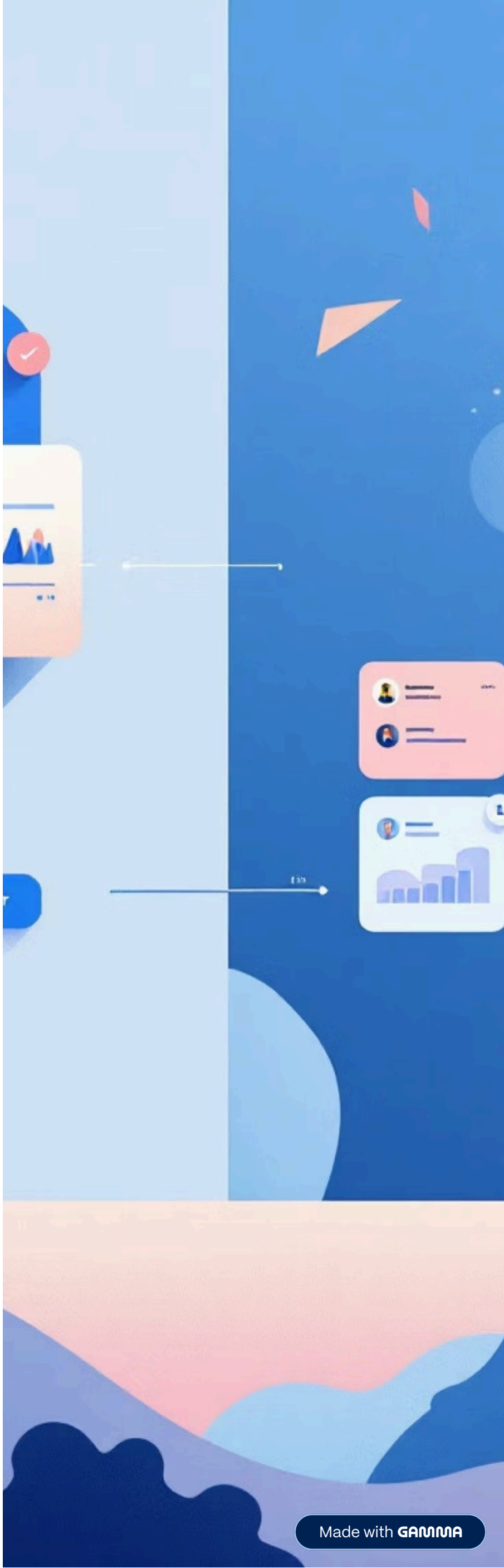
```
const UserContext = createContext();

function UserProvider({ children }) {
  const [user, setUser] = useState(null);

  return (
    <UserContext.Provider value={{ user, setUser }}>
      {children}
    </UserContext.Provider>
  );
}
```

Consume Context

```
import { useContext } from 'react';

function UserProfile() {
  const { user, setUser } = useContext(UserContext);

  return <div>Welcome, {user?.name}!</div>;
}
```

Wrap Your App

```
function App() {
 return (
 <UserProvider>
 <Header />
 <UserProfile />
 </UserProvider>
 );
}
```



Made with GAMMA

Performance Optimisation Techniques

React.memo

Prevent unnecessary re-renders by memoizing components. Wraps functional components and only re-renders when props change.

```
const ExpensiveComponent =
React.memo(({ data }) => {
  return <div>{/* Complex rendering
*/}</div>;
});
```

Lazy Loading

Load components only when needed using React.lazy and Suspense for better initial page load performance.

```
const LazyComponent = React.lazy(()
=>
  import('./MyComponent')
);

<Suspense fallback={<Loading />}>
  <LazyComponent />
</Suspense>
```

useMemo & useCallback

Optimise expensive calculations and prevent function recreation on every render, reducing child component re-renders.

```
const memoizedValue = useMemo(()
=>
 expensiveCalculation(props), [props]
);

const memoizedCallback =
useCallback(() => {
 doSomething(a, b);
}, [a, b]);
```

Made with GAMMA

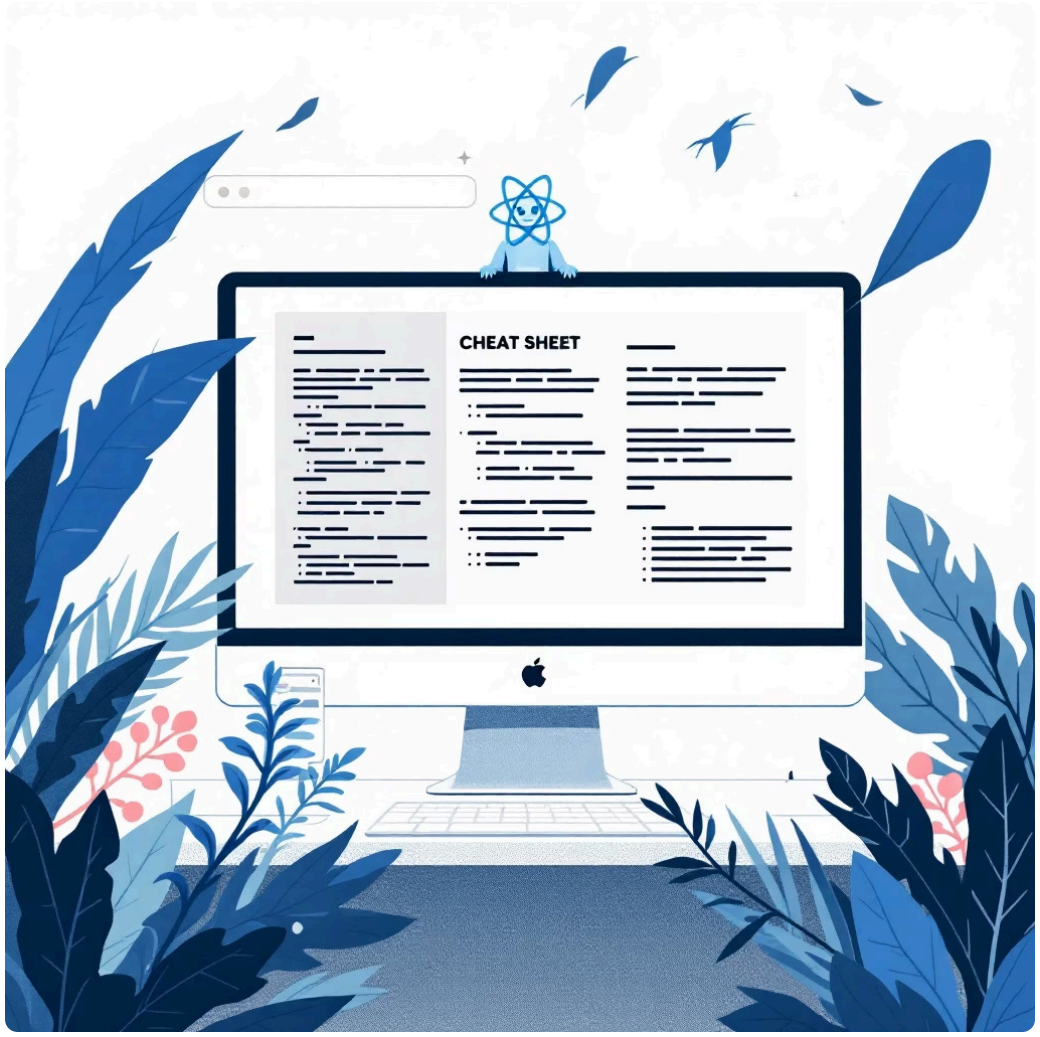
React Developer's Quick Reference

Essential Hooks Cheatsheet

Hook	Purpose
useState	Manage component state
useEffect	Side effects & lifecycle
useContext	Access context values
useRef	DOM access & mutable refs
useMemo	Memoize expensive calculations
useCallback	Memoize function references

Common Patterns

```
// Conditional Rendering
{isVisible && <Component />}
{condition ? <A /> : <B />}

// List Rendering
{items.map(item =>
  <Item key={item.id} data={item} />
)}
```



Next Steps for Mastery

- **Next.js:** Full-stack React framework
- **TypeScript:** Type-safe React development
- **Testing:** Jest + React Testing Library
- **State Management:** Redux Toolkit, Zustand
- **Styling:** Tailwind CSS, Styled Components

Practice Projects: Build a Todo app, Weather dashboard, or E-commerce site to solidify your React skills through hands-on experience.

Made with GAMMA